

ChooChoo Chores

Building a real, privacy-first app with spec-driven AI development and verify with automated AI-security-testing

A prompt → a reviewed spec → a shipped product → automated penetration testing

Sven Schleier · choochoo-chores.com

$ cat /proc/me

My name is Sven!

- Doing consulting for 15+ years in various roles in Cyber Security (focus on AppSec, Pentesting and Cloud Security)
- Worked and lived in Germany, China, Singapore and now Austria.
- Leading the Cyber Security Centre of Excellence at SoftwareOne
- Since 2016 one of the project leaders for the OWASP Mobile Application Security Project (MAS)



Labor market impacts of AI

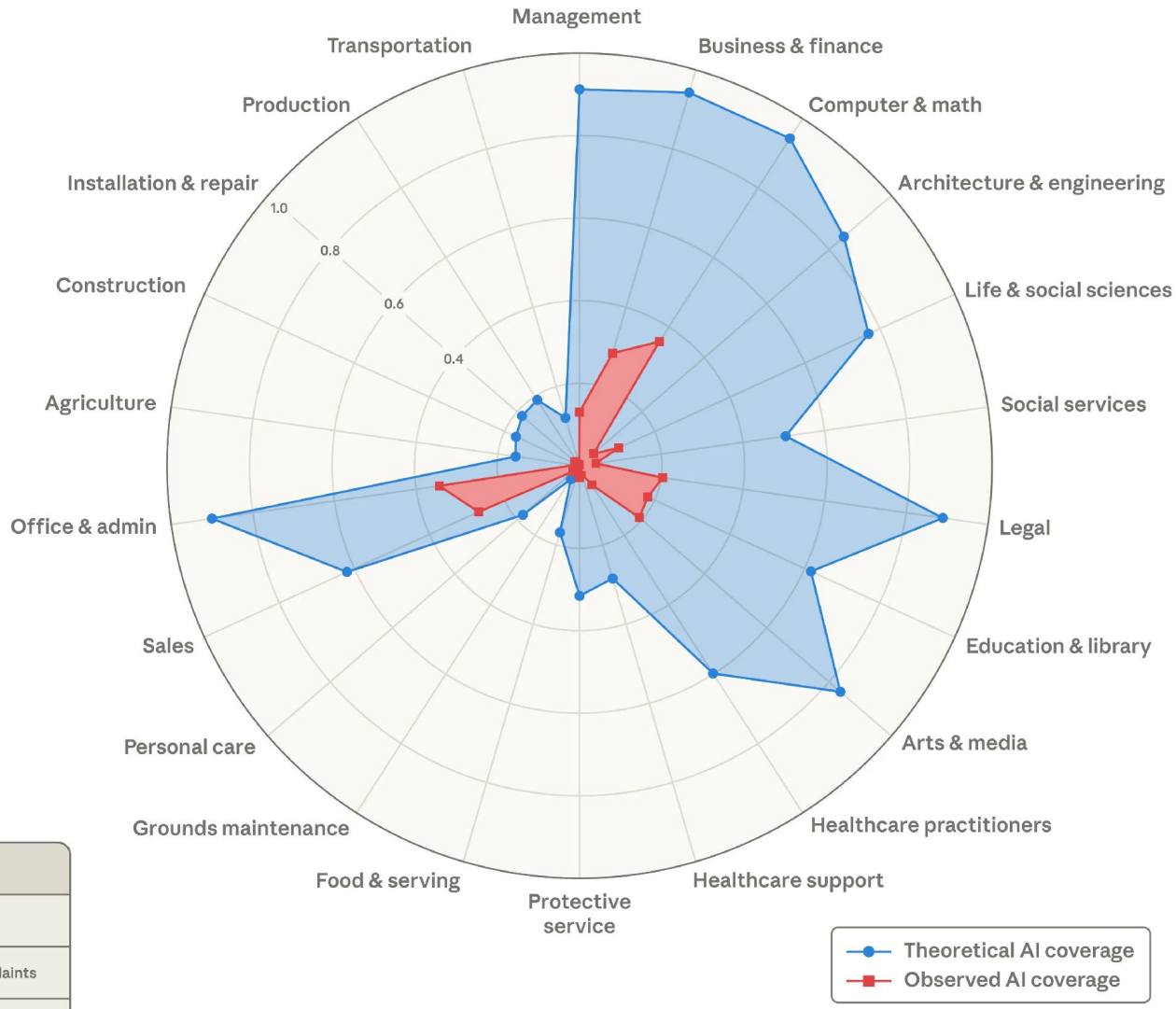
Report of Anthropic and it's key findings:

- AI is far from reaching its theoretical capability: actual coverage remains a fraction of what's feasible
- We find no systematic increase in unemployment for highly exposed workers since late 2022, though we find suggestive evidence that hiring of younger workers has slowed in exposed occupations
- Figure below shows the ten occupations most exposed to AI displacement risk. In line with other data showing that **Claude is extensively used for coding**, Computer Programmers are at the top, with 75% coverage

Most exposed occupations

Occupation	Observed exposure	Leading automated task
Computer programmers	74.5%	Write, update, and maintain software programs
Customer service representatives	70.1%	Confer with customers to provide info, take orders, handle complaints
Data entry keyers	67.1%	Read source documents and enter data into systems

Theoretical capability and observed usage by occupational category



Labor market impacts of AI

How good is AI in development, testing and fixing really?

I wanted to understand:

- Can I create a fully functional working and secure app without writing any code and following a SSDLC?
- Can I automate the deployment with a fully functional working and secure CI/CD pipeline, including IaC?
- Can I automate the penetration test to verify a secure deployment and achieve a proper testing coverage?



THE IDEA

ChooChoo Chores — chores, made rewarding

Turn everyday family chores into a calm, rewarding routine kids enjoy — and parents stay in control of.



Then: coins buy rewards, or fund a savings goal every earn & spend in a coin ledger.
Chores are one-off or recurring (daily / weekly / custom). Full DE + EN.

THE PREMISE

Vibe coding vs. spec-driven development

~ Vibe coding

- Prompt, run, eyeball, repeat
- No shared source of truth
- Fast to start, trial and error and drifts over time
- Hard to review, test or maintain
- Context lost between sessions
- Great for prototypes & throwaways

✓ Spec-driven development

- Write the spec first: requirements → design → tasks
- The spec is the source of truth; code is its output
- Slower to start — compounding returns
- Reviewable, testable, verifiable
- Context lives in the version-controlled docs, not the chat
- Built for real, maintained software

ChooChoo Chores was built spec-driven from day one — every feature traces to a written requirement.

THE LANDSCAPE

Spec-driven frameworks — and how we did it

A crowded space in 2026

Superpowers
obra/superpowers

A Claude Code skills-library plugin — composable capabilities.

GitHub Spec Kit
github/spec-kit

Opinionated toolkit with a “specify” CLI: specify → plan → tasks.

BMAD-METHOD
bmad-code-org/BMAD-METHOD

12+ specialised agents across the whole SDLC.

GSD
gsd-build/get-shit-done

~~Get Shit Done~~ now: Git. Ship. Done. — a phased spec → execute → verify flow.

OpenSpec
Fission-AI/OpenSpec

A lightweight, tool-agnostic spec format for change proposals.

Our approach

No named framework.

Claude and I simply agreed to work spec-driven — then wrote requirements.md and design.md by hand.

Those documents became the single source of truth the AIs built against.

HOW IT STARTED

A prompt, then a spec I reviewed line by line

The starting prompt

“Build a web app to manage household chores…”

- A web app to manage household chores (including mobile apps)
- Kids do chores, mark them done, get a reward (e.g. like coins?)
- Parent and child roles with separate UI and functionality
- Define rewards that can be exchanged for coins

≈ 450

lines of requirements.md

- Drafted with Claude Fable and max effort
- Reviewed line by line
- The cornerstone of the development: most functionality was already clarified & outlined here

One good prompt + a careful human review became the source of truth for everything after.

HUMAN-SUPPLIED CONTEXT

The security guidance I fed into the spec

1

OWASP ASVS / WSTG

The application-security verification baseline (L1+L2) the design is measured against and testing guide.

2

OWASP Cheat Sheets & Top 10

Patterns for AuthN/AuthZ, sessions, CSRF, error handling.

3

OWASP MASVS / MASTG

Mobile app security & testing guide.

4

AWS Well-Architected Framework + security

Least-privilege IAM, EU data residency, encrypted backups.

5

NIST 800-63B

Password guidance — length over complexity

6

GDPR / GDPR-K

Consent, export & erasure; children's data & the Play Families policy.

I hand-picked these and wrote them into the prompts — so requirements.md and design.md incorporated them from the first draft.

A small hardened stack

Client	Vue SPA (web) + Capacitor (iOS / Android) — one responsive codebase
Edge	Caddy reverse proxy — TLS · HSTS · full CSP · security headers
API	Fastify + TypeBox schemas · background jobs (pg-boss) · versioned REST
Data	PostgreSQL 16 · per-family encryption keys · tamper-evident audit log
Infra	AWS Lightsail · Docker Compose · Terraform (IaC) · EU-hosted (Frankfurt)

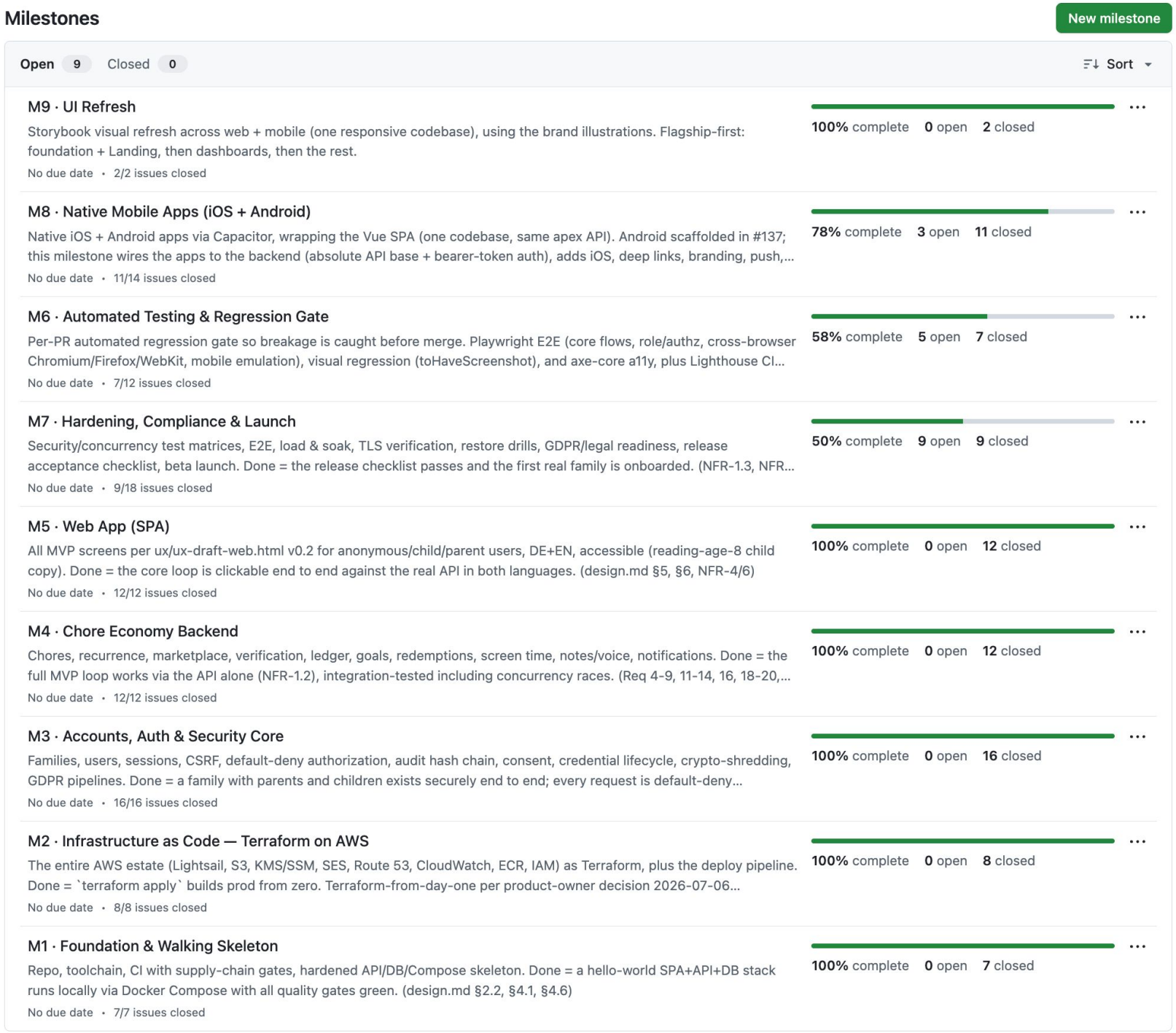
A KEY OUTPUT OF THE SPEC

Github Milestones

I told Claude to create around 6-8 milestones with no more than 20 issues per milestone.

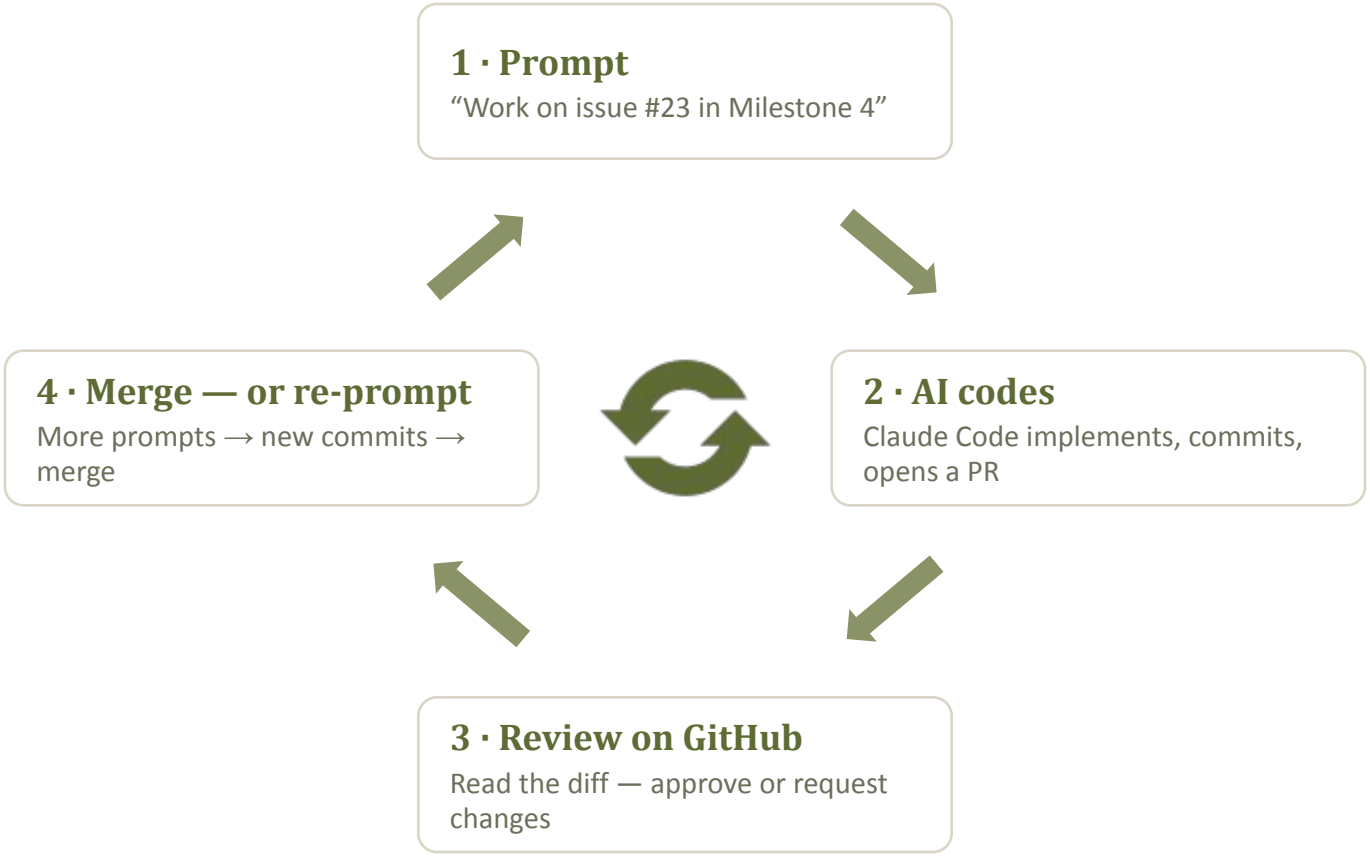
Claude created a script to populate the issues in this new Github Repo and the defined milestones we agreed on.

M9 was defined at the end of the project to create updates to the app.



HOW WE WORKED FOR CODING

The whole dev loop — no IDE required



Not a single IDE

- No VS Code, not even the Claude Code extension
- No local editor, no terminal sessions
- Chat prompts + GitHub in the browser — the entire toolchain
- Claude running in a VM

Every line of this project shipped through this loop.

Protection of repo:

- I created a new Github repo and a personal access token for Claude that gave it only access to the repo with the permissions needed.
- GitHub Branch Protection was activated so a PR needed to be created by Claude

DESIGN EXPLORATION WITH CHATGPT

Mascot



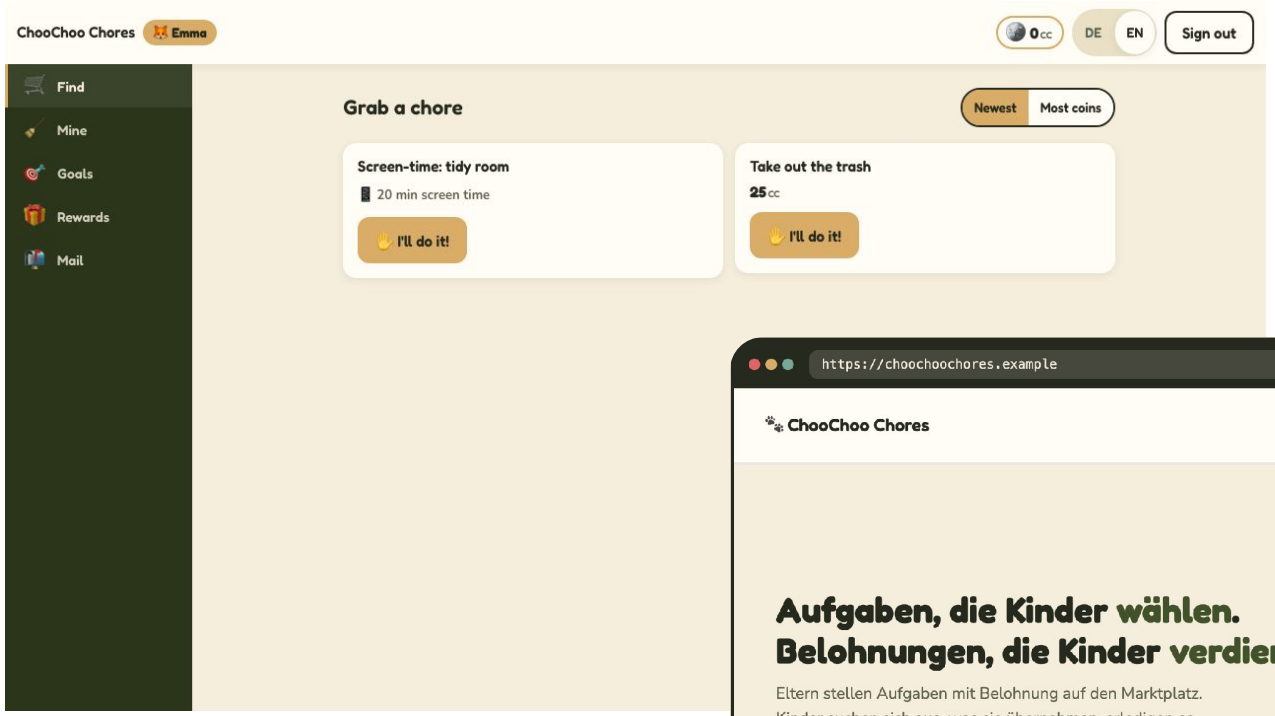


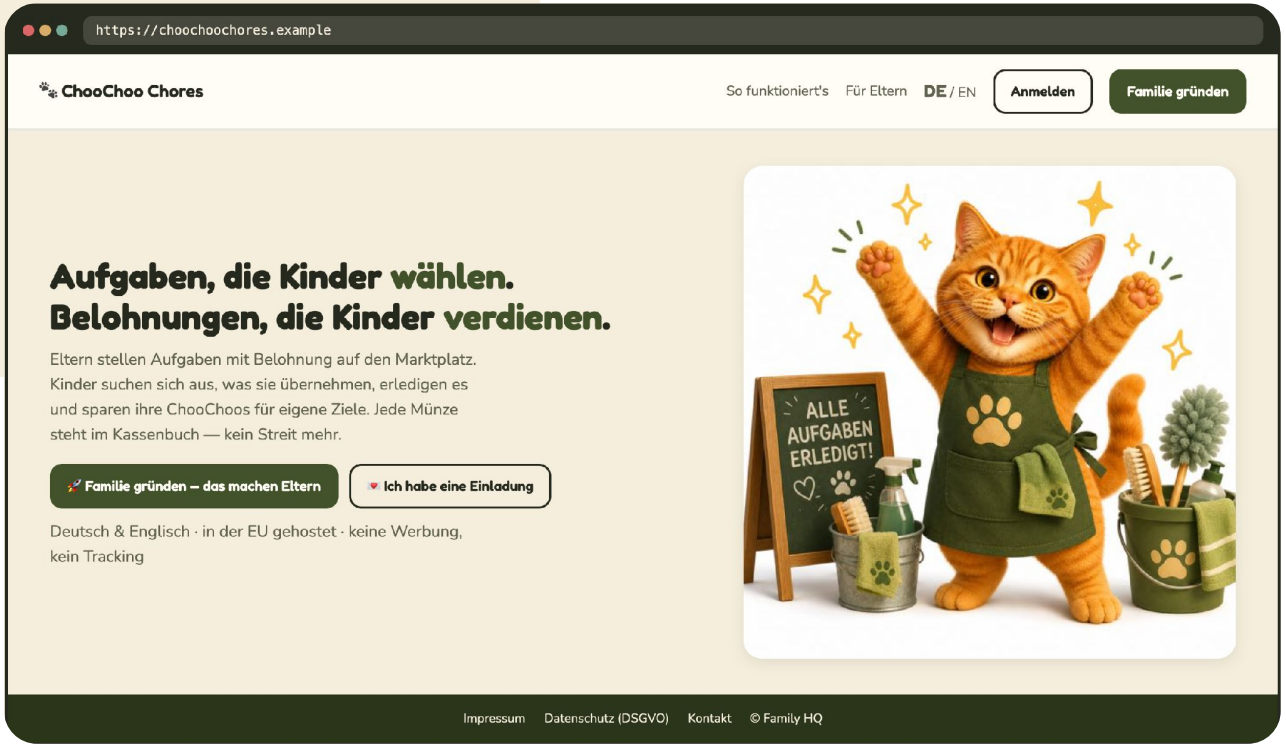






The look & the screens



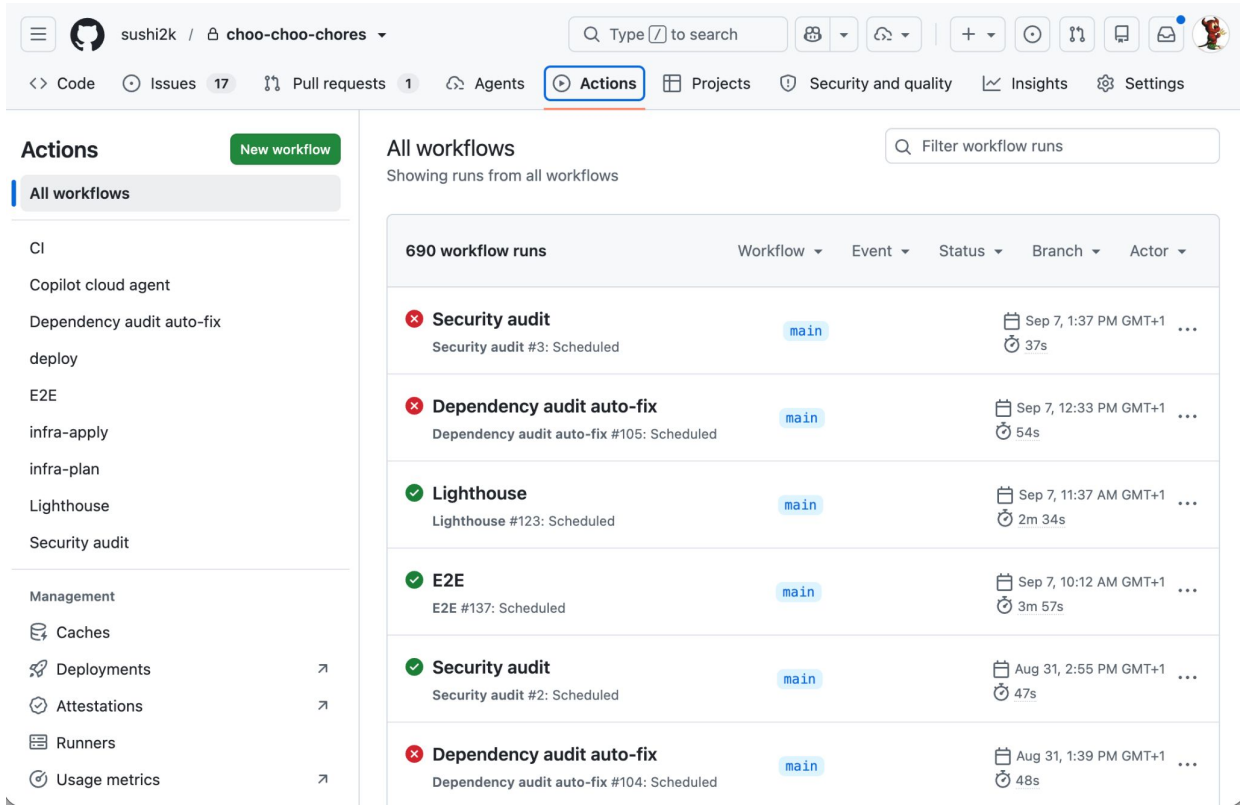


CONFIDENCE TO SHIP

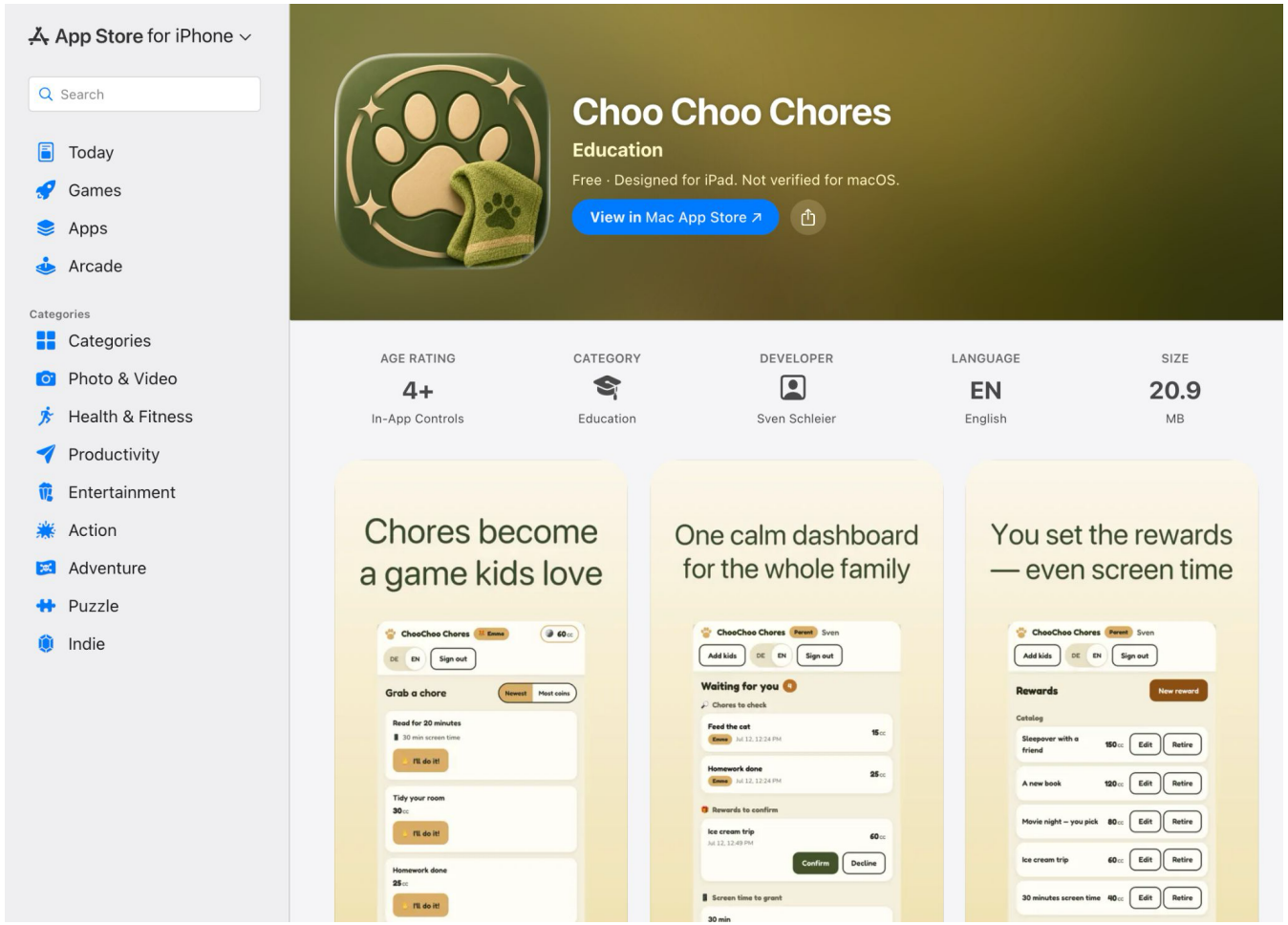
CI/CD & quality gates

GitHub Actions

- CI — lint · test · build
- E2E — Playwright browser/viewport matrix
- Lighthouse — perf-budget gate
- Dependency audit auto-fix — opens PRs for high advisories
- Infra plan / apply (Terraform, OIDC) · Deploy ⟶ ECR + Compose



iOS app done / Android in Review



https://apps.apple.com/us/app/choo-choo-chores/id6790095799

THE COLLABORATION

Built by a team of AIs — and one human

C

Claude

The spec docs, architecture, the backend & app code, tests, hardening, debugging and deploy.

G

ChatGPT

The logo, UI design, every screen and illustration — the whole visual identity.

Co

GitHub Copilot

The threat model — STRIDE + OWASP ASVS L1/L2 (PR #147).

Explored, but will not share today.

S

Sven (human)

Project structure, product decisions, accounts, store submissions, and orchestrating the three AIs.

Claude was executed in a Virtual Machine to not give access to my host file system.

Summary of spec-driven development experiences

~ What worked not so well…

- **<u>The codebase is growing. Our understanding of it isn't.</u>**
- Architecture of Github Actions was becoming complicated and excessive (burning Github Action Minutes)
- Usage limit reached -> "Put in more coins"
- Dependency on Frontier Platform Provider where future cost model and token usage is not clear (Anthropic and Open AI in this case)

✓ What worked well

- Creation of specifications and structuring them in milestones
- Not only a PoC but a working web and mobile app could be created in days that can be maintained
- Build according to security best practices, including IaC and several Github Actions for deployment, security and E2E flow testing.
- Guidance through processes (App Store / Play Store)
- Create design Ideas (UX, user flow, logos etc.)
- Fixing (small) bugs

Security testing

Pentesting — frameworks, methodology & results

Penetration Test (Pentest)

- An authorized simulated attack on your application to find exploitable weaknesses before real attackers do.
- Goes beyond automated scanners by manually verifying and chaining vulnerabilities to show real impact.
- Runs against an agreed scope of apps, APIs and environments within a fixed time window.
- Delivers a report with reproducible findings, risk ratings and fix guidance you can turn into tickets.



AI Agent Pentest Frameworks

Project	Stars*	Best suited for	Approach	License
Strix	50.5k	Web applications, APIs, CI/CD	Multi-agent recon, exploitation, PoC validation, reporting and optional fixes	Apache-2.0
Shannon	46.6k	White-box testing when you have source code	Analyzes source, attacks the running application, and reports validated exploits	AGPL-3.0
PentAGI	21.7k	Self-hosted black-box/network pentesting	Autonomous multi-agent system using tools such as Nmap, Metasploit and SQLMap in Docker	MIT
PentestGPT	14.8k	Research, CTFs and lighter CLI workflows	Staged recon → exploitation → reporting pipeline driven through Codex or Claude Code	MIT
CAI	9.7k	Building custom offensive-security agents	Extensible framework with specialized agents, tool integration, handoffs, guardrails and many LLM providers	Mixed/custom—review before commercial use

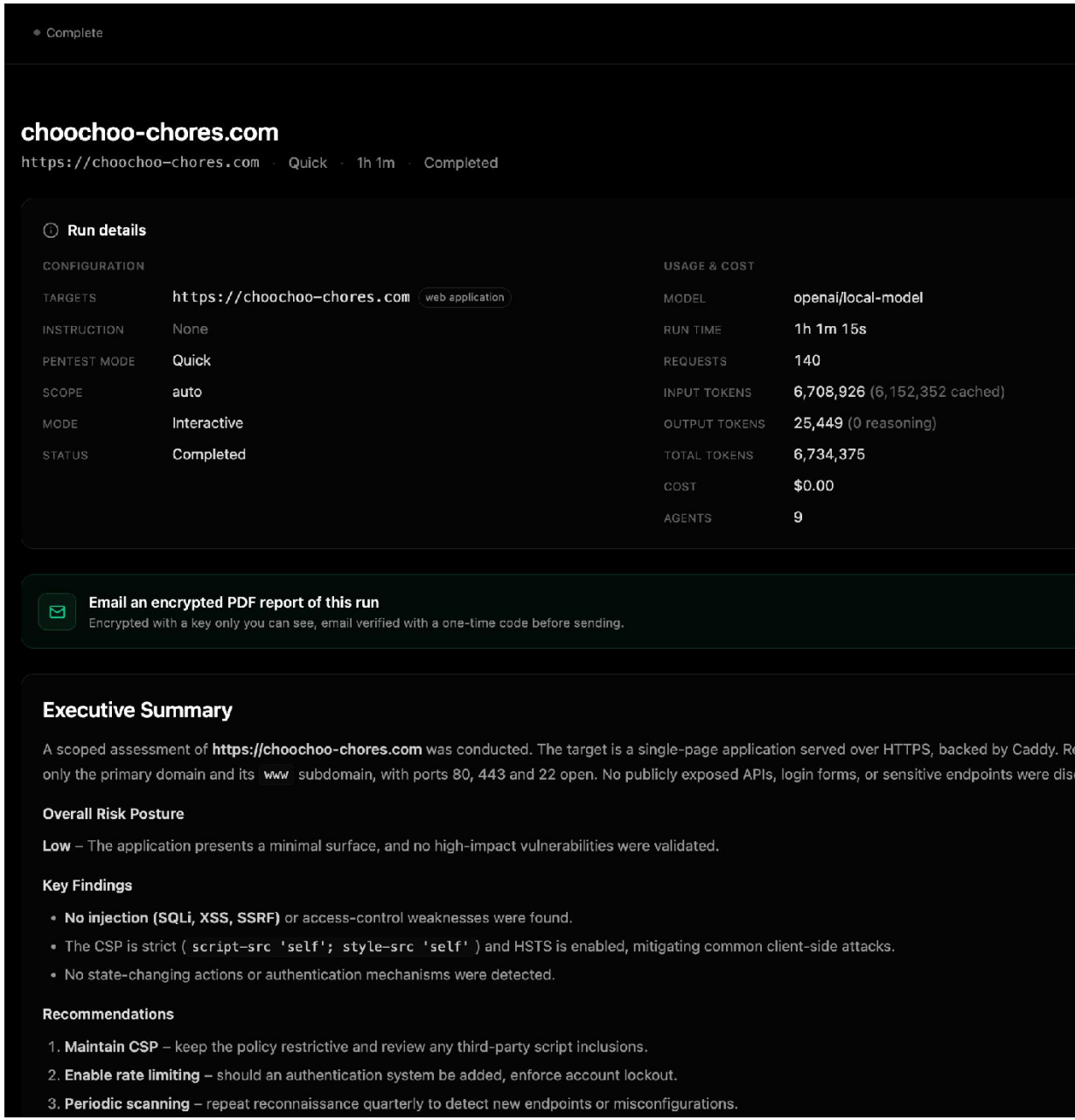
RESULTS

Strix



Local LLM via Llama.cpp
Unauthenticated – over 1 hour

```
$ STRIX_LLM=openai/local-model
$ LLM_API_BASE=http://localhost:8080/v1
$ LLM_API_KEY=sk-local
$ LLM_TIMEOUT=600
$ llama serve \
  -hf ggml-org/gpt-oss-20b-GGUF \
  --alias local-model \
  --host 0.0.0.0 --port 8080 \
  --ctx-size 65536 --parallel 1 \
  --n-gpu-layers 99 \
  --jinja \
  -fa on --cache-type-k q8_0 --cache-type-v q8_0 \
  -ub 2048 -b 2048
```



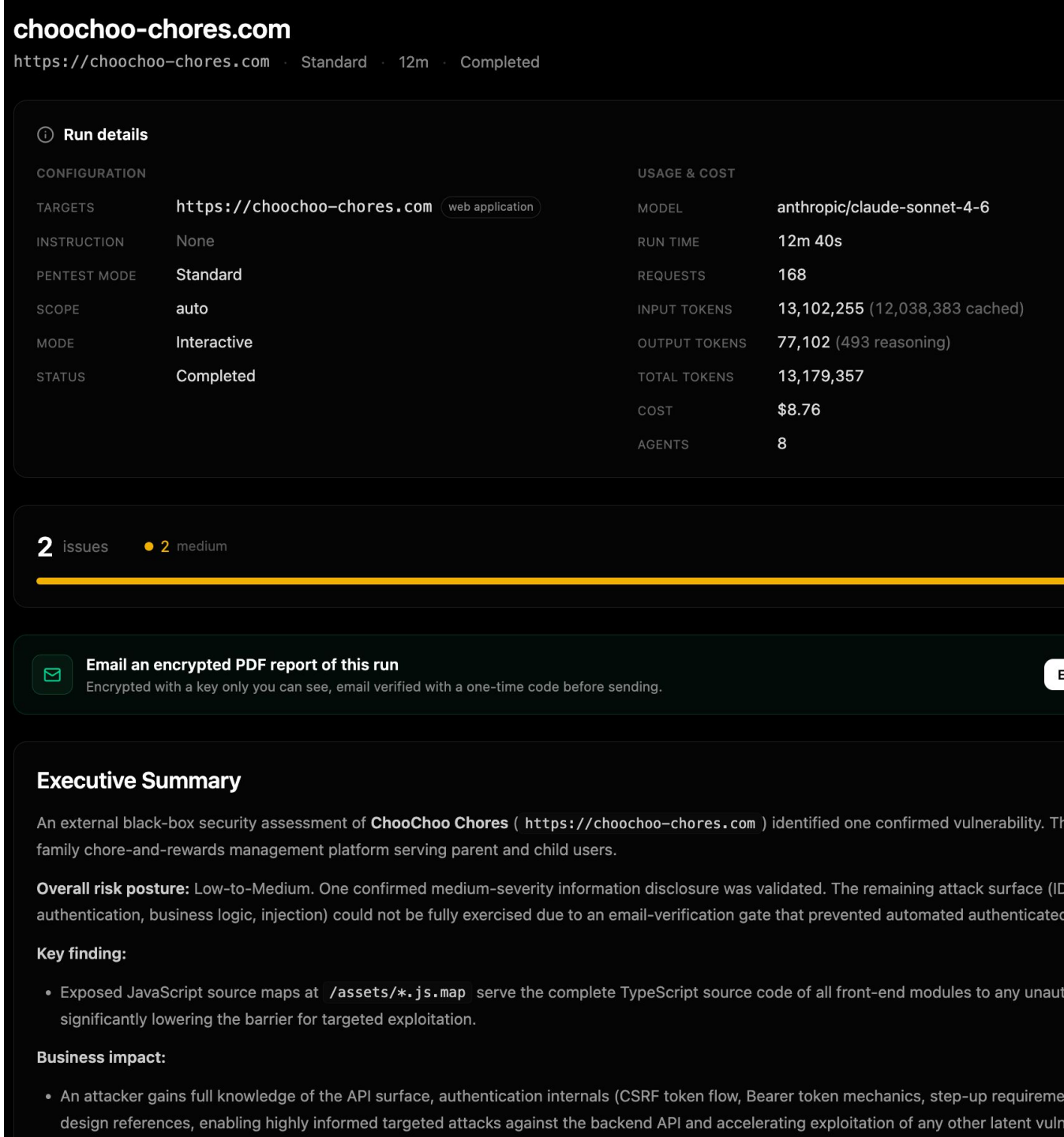
RESULTS

Strix

Frontier with Claude Console (API) unauthenticated took around 12 minutes

```
$ LLM_API_BASE=https://api.anthropic.com
$ STRIX_LLM="anthropic/claude-sonnet-4-6"
$ LLM_API_KEY="sk-ant-..."

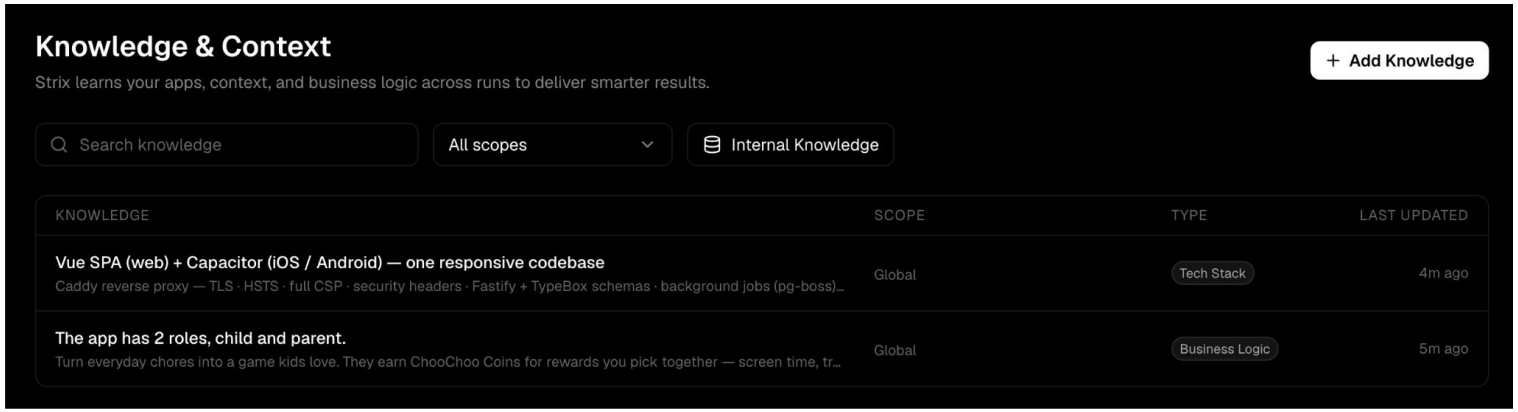
$ strix --target https://choochoo-chores.com --
scan-mode standard --max-budget 9
```

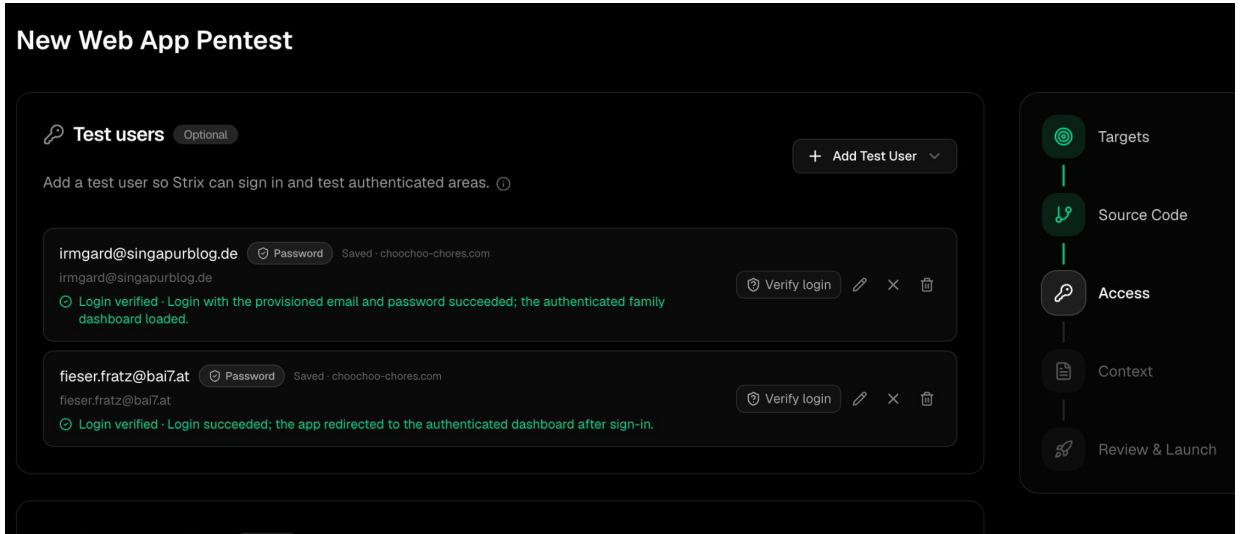


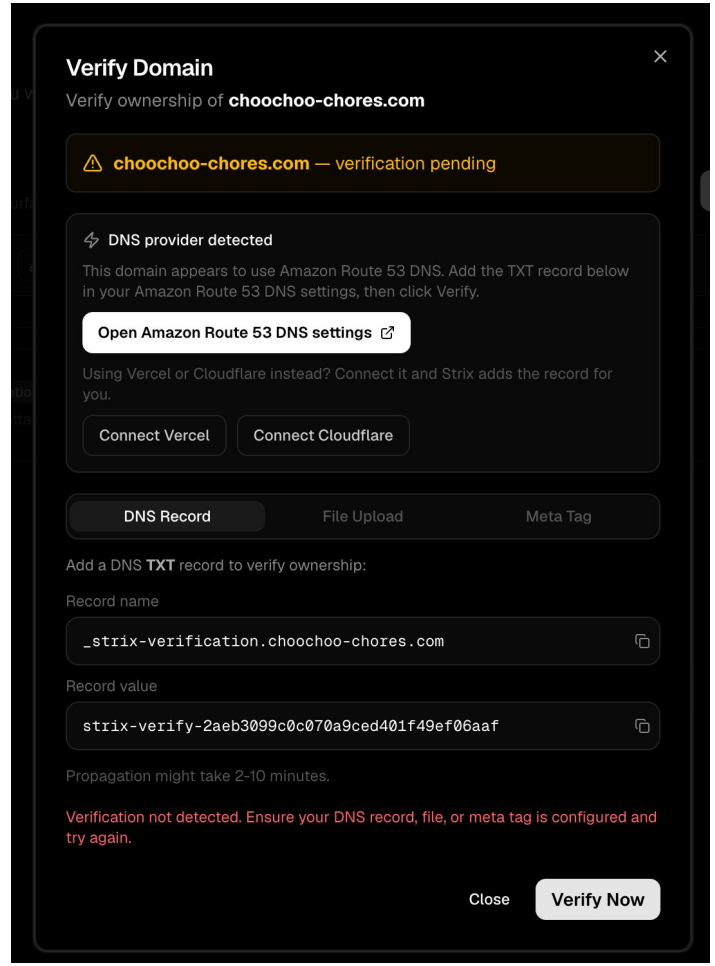
RESULTS

Strix - Cloud

Scan from the cloud for 80 USD (~70 Euro) including access to Github repo (white-box test).



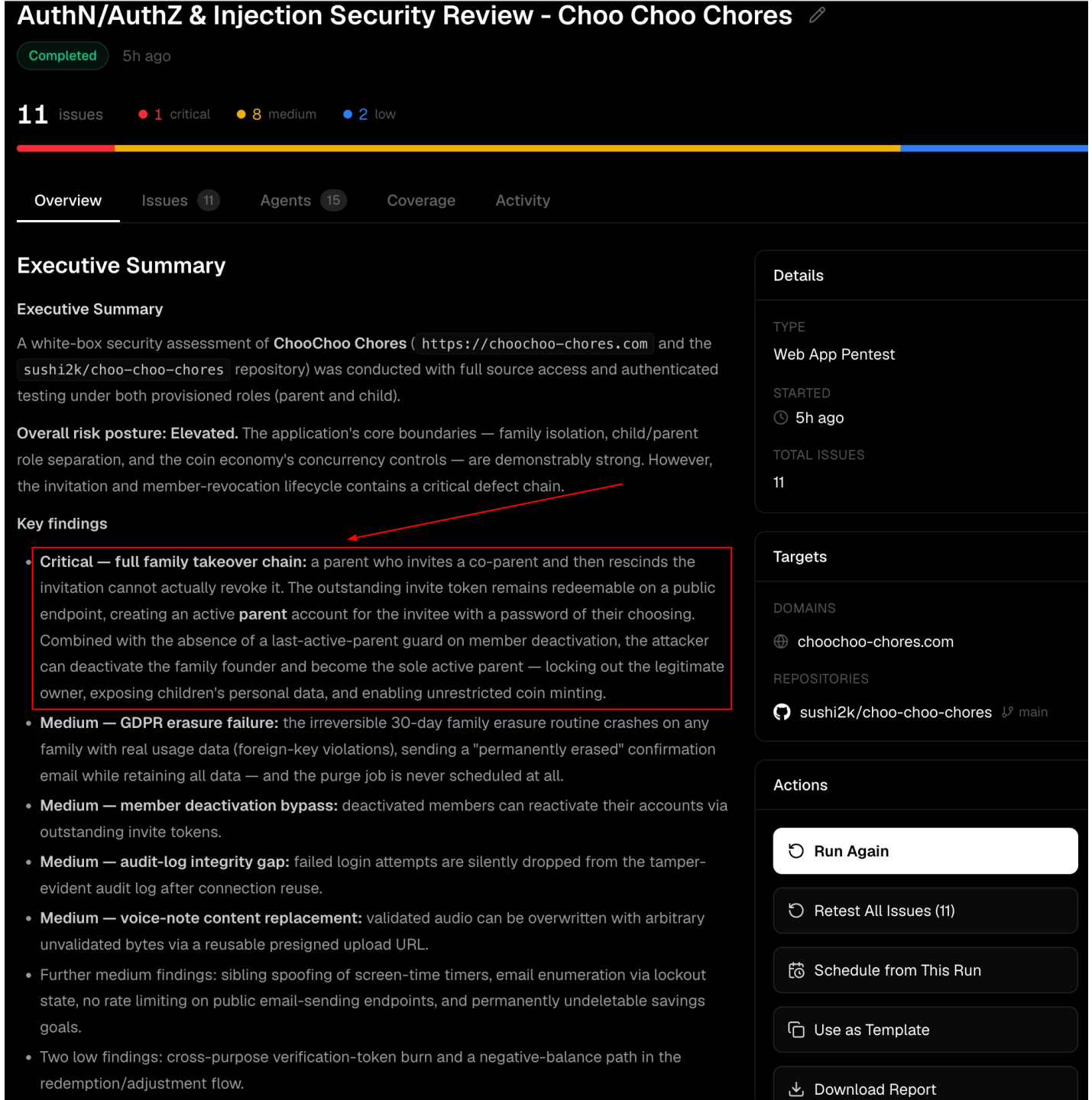




RESULTS

Strix - Cloud

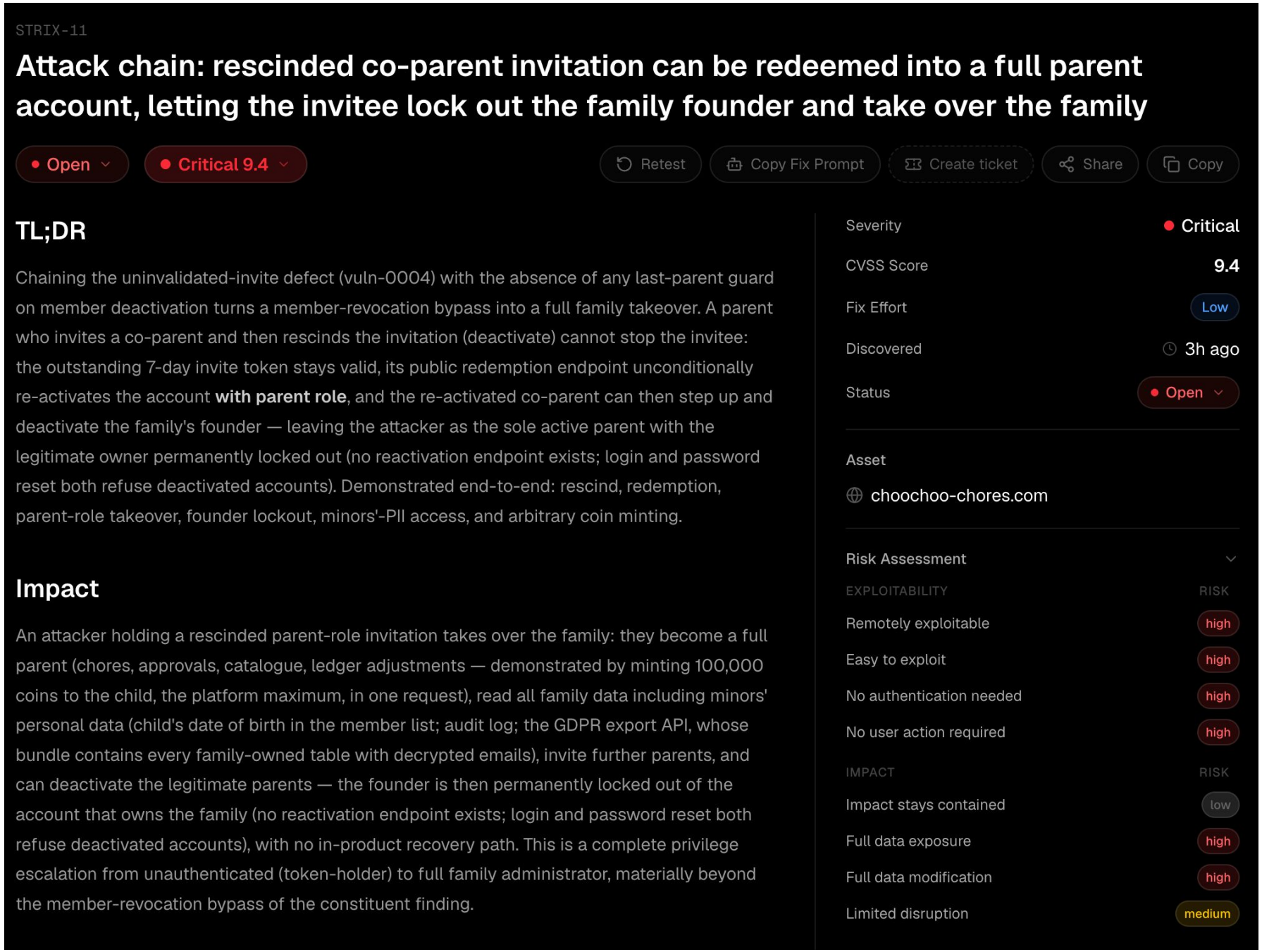
Pentest done after 2 hours



RESULTS

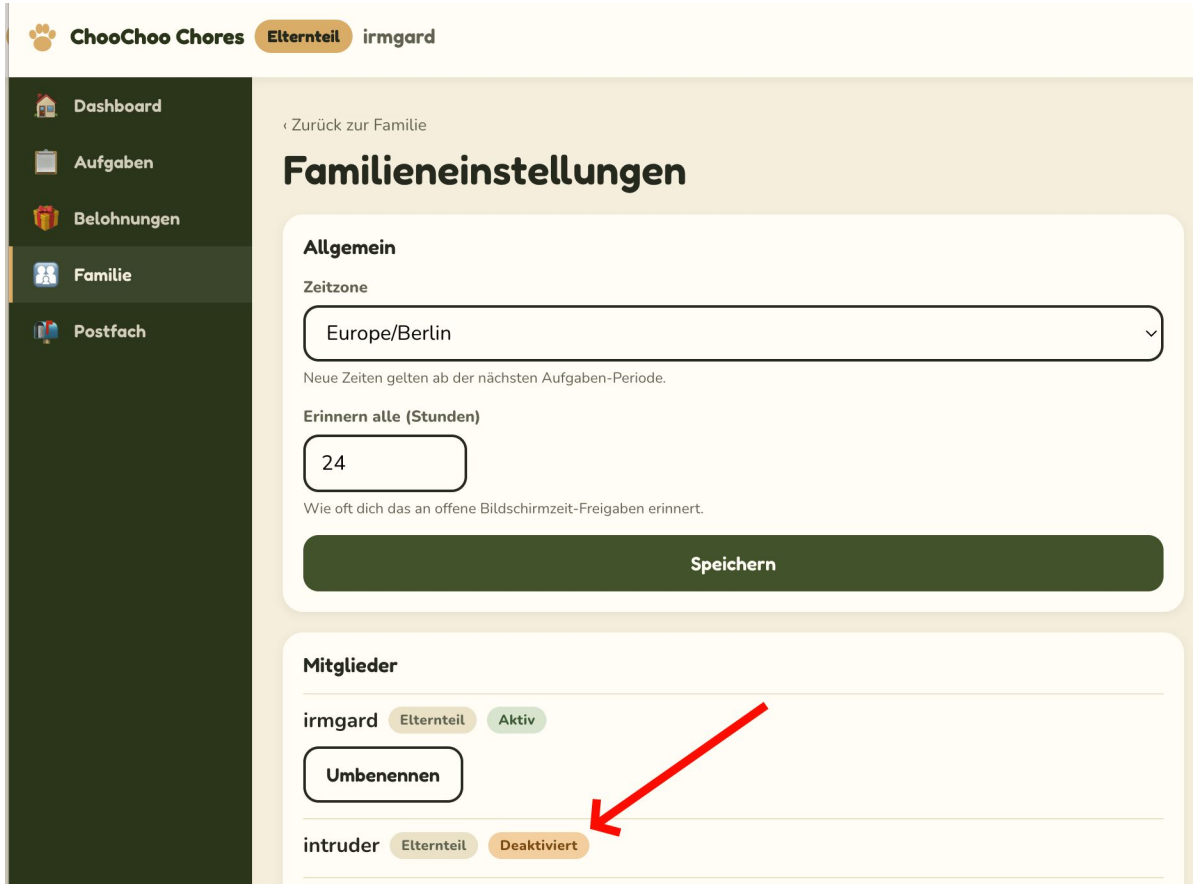
Strix - Cloud

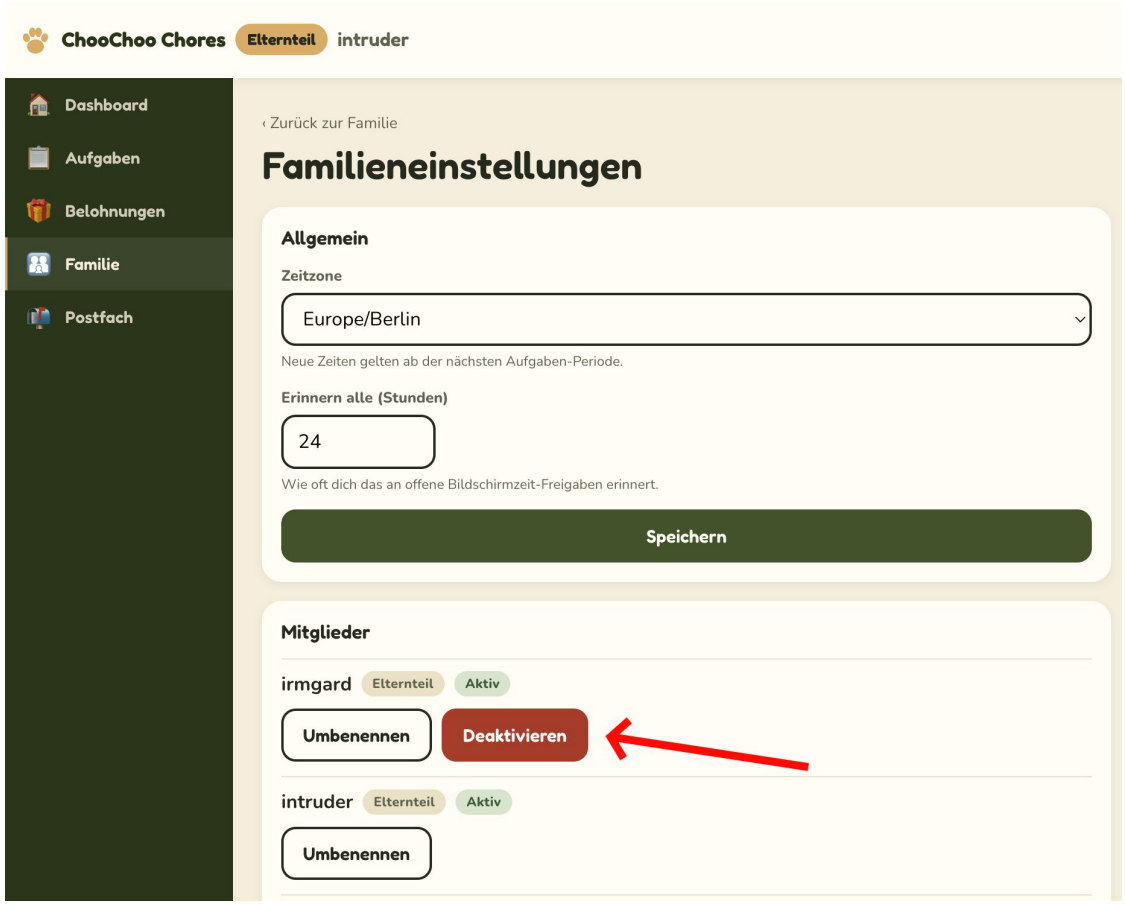
Should be considered a low-medium finding, as you need an invite and then you only have access to the family you were invited to.



Strix - Cloud

Attack works, it’s a genuine business logic vulnerability!





Lessons learned

What worked, what surprised us, and what comes next

LESSONS LEARNED

Summary



- We are going into an area where we will see a consolidation of our current tasks
- “Human/Expert in the loop is needed” - see 1Password
- Open Source Projects are defining new rules - see ZAP Proxy
- Spec-Driven development pays off
- Using AI can automate some of the monotone (but also interesting…) pieces of work
- AI is there to stay, already now people are used to work with AI and delegate tasks
- Services like AWS Continuum cost 50 USD per hour, including other AI Pentesting Services

- What if the pentest agent is leaving the target scope?
- What about Junior roles?
- Missing knowledge transfer and SPOF in companies, if there are 100x Engineers. But what do companies do when they leave?
- Will the costs for token exceed human labour costs?
- How will the role of the pentester change?
    - “Pentest Orchestrator/Operator”
    - ”Harness Engineer”

What's next for me?

- **Threat Modeling** Using AI-assisted Threat Modeling
- **SAST** Using AI-assisted Source Code Review
- **Build more CI security gates with AI** Secret scanning, container/IaC CVE scanning
- **Mobile App Testing** AI-Assisted testing for iOS/Android

What's next for our friends from the frontier companies?





From family chores to a shipped, privacy-first app

Spec-driven, built and tested with a team of AIs — and one human in the loop.

Sven Schleier · Bai7 GmbH · choochoo-chores.com

Thank you